

eClinician – Architectural Analysis and High-Level System Architecture

Course: CS425 – Software Engineering

Assignment: Lab 3 – Architectural Analysis

Student: Hadijah Kyampeire

Registration #: 618990

1. Introduction

The eClinician system is a web-based clinical management application designed to digitize the daily operations of healthcare facilities. The system enables healthcare professionals to manage patient registration, appointments, medical records, prescriptions, laboratory requests, laboratory results, and user accounts through a centralized platform.

The Vision document established the overall goals of the project, while the Software Requirements Specification (SRS) identified the functional requirements through a use-case-driven analysis. Based on these requirements, this document presents the first iteration of the architectural analysis and proposes an initial high-level system architecture.

The objective of the architecture is to provide a secure, maintainable, scalable, and extensible foundation that supports current functional requirements while allowing future enhancements such as Software-as-a-Service (SaaS) deployment and Artificial Intelligence (AI) capabilities.

2. Architectural Drivers

The selected architecture is influenced by several functional and non-functional requirements identified during the analysis phase.

Functional Drivers

The architecture must support:

- Patient Management
- Appointment Management
- Medical Record Management
- Prescription Management
- Laboratory Management
- User Management
- AI-assisted consultation summary generation

Non-Functional Drivers

The architecture should provide:

- Security
- Maintainability
- Scalability
- Reliability
- Performance
- Data Integrity
- Extensibility

These quality attributes guided the selection of the overall system architecture.

3. Selected Architectural Style

The eClinician system adopts a **Three-Tier Layered Architecture**, consisting of:

- Presentation Tier
- Application Tier
- Data Tier

Within the application tier, the system further follows a layered architecture by separating responsibilities into presentation controllers, security services, business services, AI services, and data access repositories.

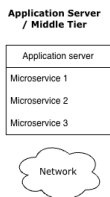
This architecture promotes loose coupling between system components, making the application easier to develop, test, maintain, and extend over time.

The architecture also aligns well with Spring Boot best practices and modern enterprise web application development.

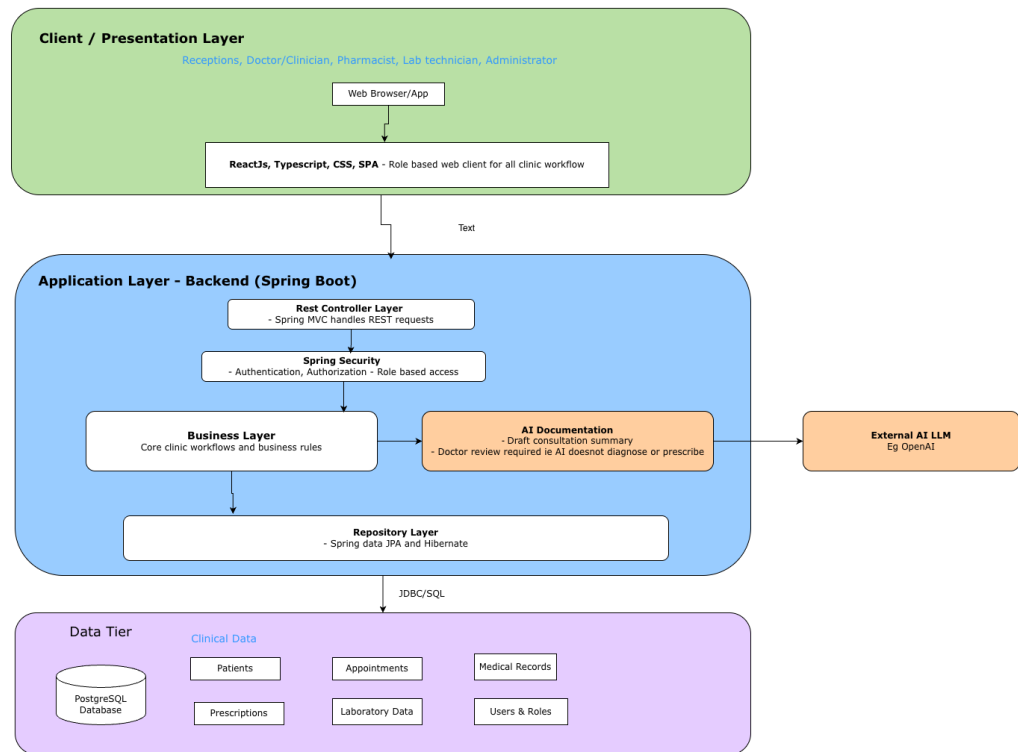
4. High-Level System Architecture

Figure 1. High-Level System Architecture for eClinician

Physical Layer



Logical layer



5. Architectural Components

5.1 Presentation Tier

The Presentation Tier provides the user interface through a web browser.

It is implemented using: React, TypeScript, HTML5, CSS, Vite

Different dashboards are presented depending on the authenticated user's role.

The supported users include:

- Receptionist
- Doctor / Clinician
- Pharmacist
- Laboratory Technician
- Hospital Administrator

The frontend communicates with the backend through secure HTTPS REST APIs using JSON.

5.2 Application Tier

The Application Tier is implemented using Java Spring Boot.

It contains several logical layers.

REST Controller Layer:

- Receives HTTP requests from the frontend.

Responsibilities include:

- Request routing
- Input validation
- Returning JSON responses

Security Layer:

Spring Security is responsible for:

- Authentication
- Authorization
- Role-Based Access Control (RBAC)
- Session/JWT management

This ensures users only access functions permitted by their assigned role.

Business Service Layer:

This layer implements the core business logic.

Responsibilities include:

- Patient registration rules
- Appointment scheduling validation
- Medical record management
- Prescription processing
- Laboratory workflow
- User management
- Business rule enforcement

AI Assistance Service:

The first AI capability introduced into eClinician is an AI-Assisted Consultation Summary Generator.

After a clinician records consultation notes, they may request the system to generate a concise clinical summary.

The AI service:

- prepares the prompt,
- Securely communicates with an external Large Language Model (LLM), validates the generated response.
- Returns the draft summary to the clinician.

The clinician must review, edit if necessary, and approve the generated summary before it becomes part of the patient's medical record.

The AI service does not diagnose illnesses, prescribe medication, recommend treatments, or order laboratory tests.

Its purpose is to improve documentation efficiency while ensuring clinicians remain responsible for all medical decisions.

Repository Layer:

The Repository Layer uses:

- Spring Data JPA
- Hibernate ORM

Responsibilities include:

- CRUD operations
- Database transactions
- Object-relational mapping
- Query execution

5.3 Data Tier

The Data Tier uses PostgreSQL as the primary relational database.

The database stores:

- Users
- Roles
- Patients
- Appointments

- Medical Records
- Prescriptions
- Laboratory Requests
- Laboratory Results
- Approved AI-generated consultation summaries
- Audit information

Database constraints ensure referential integrity and prevent duplicate or inconsistent records.

6. Communication Between Tiers

The communication between system components follows a layered approach.

- Users interact with the React web application through a browser.
- The frontend sends HTTPS REST requests to Spring Boot.
- Spring Security authenticates and authorizes the request.
- Business Services execute application workflows.
- When requested, the AI Assistance Service securely communicates with an external Large Language Model.
- Repository components persist validated information into PostgreSQL.
- JSON responses are returned to the frontend for presentation.

This layered communication minimizes coupling while improving maintainability and security.

7. Architectural Decisions

The following technologies were selected.

Component	Technology	Reason
Frontend	React, Typescript, CSS	Modern SPA framework with strong component architecture
Backend	Java Spring Boot	Enterprise framework with excellent REST and security support
Security	Spring Security	Authentication and role-based authorization
Database	PostgreSQL	Reliable relational database with strong transactional support
ORM	Spring Data JPA, Hibernate	Simplifies persistence and database access
AI	External LLM API	Enables AI-assisted documentation while keeping clinical decisions under human supervision

8. Quality Attributes

The selected architecture supports several important software quality attributes.

Security:

- Role-based authorization ensures users only access features appropriate to their responsibilities.

Maintainability:

- Layer separation reduces dependencies and simplifies future modifications.

Scalability:

- The presentation, application, database, and AI components can be scaled independently.

Reliability:

- Database constraints and business rules maintain consistent healthcare records.

Performance:

- REST APIs and a React Single-Page Application reduce unnecessary page reloads and improve responsiveness.

Extensibility:

- The modular architecture allows future integration of additional AI services, reporting modules, and multi-clinic SaaS capabilities without major redesign.

9. Conclusion

The proposed architecture provides a strong foundation for the implementation of the eClinician system. The three-tier layered architecture separates presentation, business logic, artificial intelligence services, and data persistence into well-defined components, resulting in a solution that is secure, maintainable, and scalable.

The introduction of an AI-assisted consultation summary service demonstrates how modern AI capabilities can improve clinical documentation while ensuring that healthcare professionals remain responsible for diagnosis and treatment decisions. Furthermore, the modular design enables the system to evolve toward future SaaS deployment and additional intelligent healthcare features.